

Step 2 — Create ZCL_PO_LONGTEXT_HANDLER

SE24 Class Builder — Field-by-Field Creation Guide

Handler Class: reads STXL via READ_TEXT · writes plain rows to ZPO_LONGTEXT

TDID=F01 | Language=D | EKPO TDNAME=no space | 2026-06-28

What This Class Does

ZCL_PO_LONGTEXT_HANDLER is the engine of the PO Long Text persistency solution. It has two methods: WRITE_TO_PERSISTENCE reads the compressed text from STXL using SAP function module READ_TEXT, then writes each plain line as a row into ZPO_LONGTEXT. READ_FROM_PERSISTENCE does a simple SELECT from ZPO_LONGTEXT — no READ_TEXT needed.

Method	Level	Visibility	Purpose
WRITE_TO_PERSISTENCE	Static	Public	Builds TDNAME → calls READ_TEXT → deletes old rows → inserts plain lines to ZPO_LONGTEXT
READ_FROM_PERSISTENCE	Static	Public	SELECT * FROM ZPO_LONGTEXT WHERE EBELN = ... ORDER BY LINE_COUNTER

■ EKPO TDNAME — System-Specific (confirmed from STXH): EKKO: lv_tlname = iv_ebeln → e.g. 4500077744 EKPO: CONCATENATE iv_ebeln iv_ebelp → e.g. 450007774400010 (NO space) Standard SAP documentation says EBELN + SPACE + EBELP — NOT valid for this system.

A

Open SE24 and Create the Class

Transaction SE24 — Class Builder

1

Open SE24

Transaction code: SE24 → press Enter

2

Enter class name

Object Type: select radio button 'Class/Interface' Object Name: ZCL_PO_LONGTEXT_HANDLER Click Create button (or press F5)

3

Fill class properties

Description: PO Long Text Handler — Read and Write Persistency Instantiation: Public Final: ✓ tick (prevents subclassing) Class Type: Regular Class Click Save

4

Assign package and transport

Package: ZMM (or your development package) Transport: assign to your active workbench transport Save — class is created in New status

B

Add Method: WRITE_TO_PERSISTENCE

Methods tab — Static Public method with 5 importing parameters

1

Go to Methods tab

Click the Methods tab at the top of the SE24 screen

2

Add method row

Click in the empty row under 'Method' column and enter: Method: WRITE_TO_PERSISTENCE Level: Static Method Visibility: Public Description: Write text lines to ZPO_LONGTEXT

3

Open Parameters for this method

With the WRITE_TO_PERSISTENCE row selected (highlighted blue) Click the Parameters button in the toolbar at the top The Parameters screen opens

Parameters for WRITE_TO_PERSISTENCE

Add one row per parameter. For each: enter Parameter name → select Type (Importing) → select Typing Method (Type) → enter Associated Type name.

Parameter	Type	Typing Method	Associated Type	Default	Purpose
IV_EBELN	Importing	Type	EBELN	—	Purchase Order Number
IV_EBELP	Importing	Type	EBELP	00000	PO Item — default 00000 for header texts
IV_TDOBJECT	Importing	Type	TDOBJECT	—	Text Object: EKKO or EKPO
IV_TDID	Importing	Type	TDID	—	Text ID: F01 (confirmed for this system)
IV_TDSPRAS	Importing	Type	TDSPRAS	—	Language key: D

To set default value for IV_EBELP: Click the IV_EBELP row → Default Value column → enter: 00000 This means callers can omit IV_EBELP for header texts (defaults to 00000 = header)

4

Press F3 or Back

Press F3 to return to the Methods tab after entering all 5 parameters

5

Save

Ctrl+S to save the method definition

C

Add Method: READ_FROM_PERSISTENCE

Methods tab — Static Public method with 1 importing + 1 returning parameter

1

Add second method row

On the Methods tab, click next empty row and enter: Method: READ_FROM_PERSISTENCE Level: Static Method Visibility: Public Description: Read text lines from ZPO_LONGTEXT

2

Open Parameters

With READ_FROM_PERSISTENCE row selected → click Parameters button

3

Add parameters

Add two rows:

Parameter	Type	Typing Method	Associated Type	Default	Purpose
IV_EBELN	Importing	Type	EBELN	—	Purchase Order Number to read
RT_LINES	Returning	Type (Ref)	TABLE OF ZPO_LONGTEXT	—	Returns all text rows for this PO

For RT_LINES Returning parameter: Typing Method: select 'Type' Associated Type: enter TABLE OF ZPO_LONGTEXT Pass By Value: ✓ tick (Returning parameters must be passed by value)

4

Press F3

Return to Methods tab

5

Save

Ctrl+S

D

Implement WRITE_TO_PERSISTENCE — Full Code

Double-click method name → source editor opens → paste code

1

Open method source editor

Methods tab → double-click WRITE_TO_PERSISTENCE Source code editor opens showing METHOD ... ENDMETHOD skeleton

2

Replace the skeleton with the full code below

Select all existing content → delete → paste the code exactly as shown

```
METHOD write_to_persistence.

DATA: lt_lines      TYPE TABLE OF tline,
      lv_tlname     TYPE tdname,
      lv_counter    TYPE numc5,
      ls_persist    TYPE zpo_longtext.

" Build TDNAME – system-specific: NO space between EBELN and EBELP for EKPO
" Confirmed from STXH: EKPO TDNAME = 450007774400010 (no space)
IF iv_tdoobject = 'EKK0'.
    lv_tlname = iv_ebeln.                " Header: TDNAME = PO number only
ELSE.
    CONCATENATE iv_ebeln iv_ebelp INTO lv_tlname. " Item: no space
ENDIF.

" Read and decompress text from STXH/STXL via SAP standard READ_TEXT
CALL FUNCTION 'READ_TEXT'

EXPORTING
    id          = iv_tdid
    language    = iv_tdspras
    name        = lv_tlname
    object      = iv_tdoobject

TABLES
    lines       = lt_lines

EXCEPTIONS
    OTHERS      = 4.
```

```

" If no text found – exit cleanly, nothing to write

IF sy-subrc <> 0 OR lt_lines IS INITIAL.

    RETURN.

ENDIF.

" Full refresh: delete all existing rows for this exact text

" This ensures deleted lines are removed (not just updated)

DELETE FROM zpo_longtext

WHERE ebeln      = iv_ebeln

    AND ebelp     = iv_ebelp

    AND tdoobject = iv_tdoobject

    AND tdid      = iv_tdid

    AND tdspras   = iv_tdspras.

" Insert one flat row per plain text line

LOOP AT lt_lines INTO DATA(ls_line).

    ADD 1 TO lv_counter.

    CLEAR ls_persist.

    ls_persist-ebeln      = iv_ebeln.

    ls_persist-ebelp      = iv_ebelp.

    ls_persist-tdobject   = iv_tdoobject.

    ls_persist-tdid       = iv_tdid.

    ls_persist-tdspras    = iv_tdspras.

    ls_persist-line_counter = lv_counter.

    ls_persist-tdformat    = ls_line-tdformat.

    ls_persist-tdline      = ls_line-tdline.    " PLAIN TEXT – no compression

    ls_persist-migrated_at = sy-datum.

    ls_persist-migrated_by = sy-uname.

    ls_persist-source      = 'D'.                " D = Dual-write by BAdI

    INSERT zpo_longtext FROM ls_persist.

ENDLOOP.

ENDMETHOD.

```

3

Save the method

Ctrl+S → save after pasting — do NOT activate yet

E

Implement READ_FROM_PERSISTENCE — Full Code

Double-click method name → paste code → simple SELECT

1

Open method source editor

Methods tab → double-click READ_FROM_PERSISTENCE Source code editor opens

2

Replace with code below

Select all → delete → paste:

```
METHOD read_from_persistence.  
  
    " Direct SQL on ZPO_LONGTEXT – no READ_TEXT needed  
    " Plain text in TDLINE – no decompression  
  
    SELECT * FROM zpo_longtext  
  
        INTO TABLE @rt_lines  
  
        WHERE ebeln = @iv_ebeln  
  
        ORDER BY tdoobject, tddid, tdspras, line_counter.  
  
ENDMETHOD.
```

3

Save

Ctrl+S

F

Activate the Class and Verify

Ctrl+F3 — both methods must turn blue (Active)

1

Activate the class

Press Ctrl+F3 (Activate) OR Menu: Class → Activate If syntax errors appear → check code matches exactly and ZPO_LONGTEXT is Active in SE11

2

Verify activation

Status bar shows: Object ZCL_PO_LONGTEXT_HANDLER activated Methods tab: both method names turn BLUE = active Top of screen shows: Active (not Revised or New)

3

Quick syntax test

SE38 → new test program → add this code and run: `zcl_po_longtext_handler=>write_to_persistence( iv_ebeln = '4500077744' iv_ebelp = '00000' iv_tdoobject = 'EKKO' iv_tdid = 'F01' iv_tdspras = 'D' )`. Then check SE16N → ZPO_LONGTEXT → filter EBELN=4500077744 If rows appear → handler is working correctly ✓

■ Step 2 complete when: SE24 → ZCL_PO_LONGTEXT_HANDLER → Status = Active Both methods show blue in Methods tab SE16N → ZPO_LONGTEXT → rows appear after test call (2 rows for 2 text lines)

Quick Reference — Both Methods

Method	Called By	Input	What It Does	Output
WRITE_TO_PERSISTENCE	BAdI (Step 3) Migration program	EBELN, EBELP, TDOBJECT, TDID, TDSRAS	Calls READ_TEXT → decompresses STXL → DELETE old rows → INSERT plain rows	Rows in ZPO_LONGTEXT with SOURCE=D
READ_FROM_PERSISTENCE	BDC programs Reports CDS views	EBELN only	SELECT * FROM ZPO_LONGTEXT WHERE EBELN = ... ORDER BY LINE_COUNTER	Table of ZPO_LONGTEXT rows — plain readable text

After Step 2 is complete — proceed to: Step 3: SE19 → Create ZCL_PO_TEXT_BADI_IMPL BAdI ME_PROCESS_PO_CUST fires on every ME21N/ME22N save Calls ZCL_PO_LONGTEXT_HANDLER=>WRITE_TO_PERSISTENCE automatically Full BAdI code is in: PO_LongText_4Steps_Guide.pdf → Step 3 section